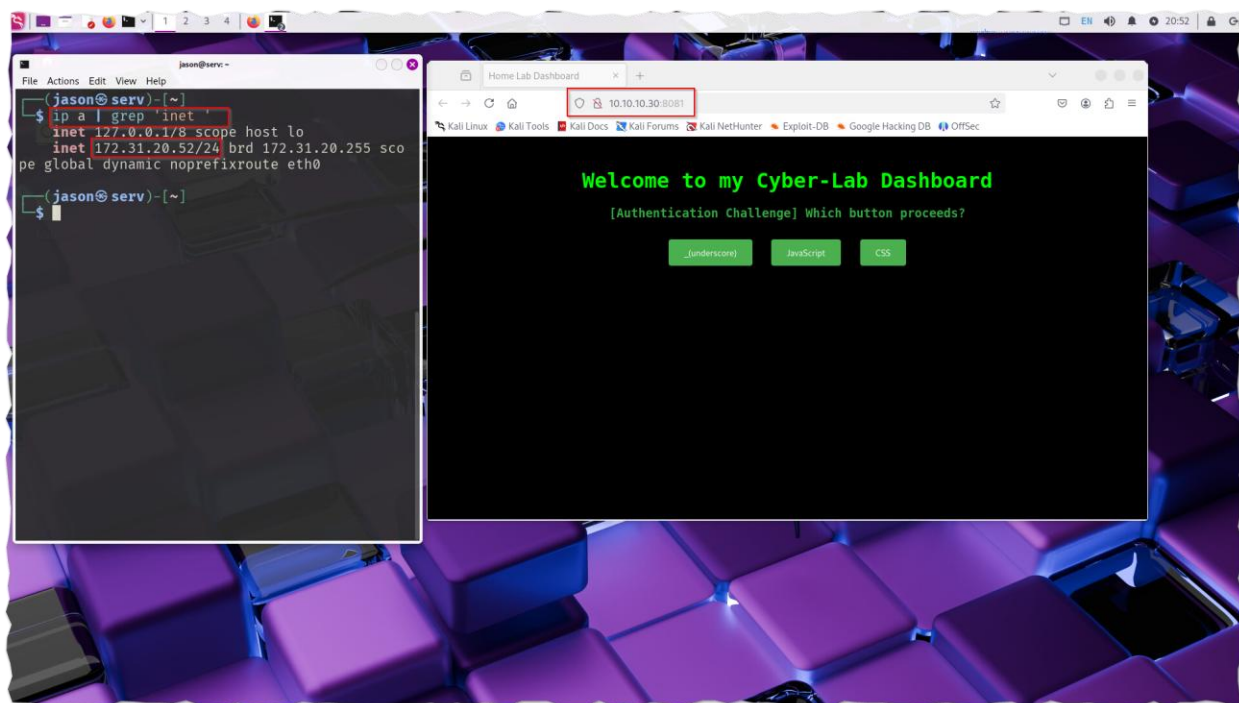




✅ Lab Recap — Screened Subnet Web App Access (pfSense + VLAN)

Objective

Create a **screened subnet (VLAN)** that is **isolated by default**, then selectively allow **access to a single internal web application** hosted on a Raspberry Pi — without exposing the rest of the LAN.

What Was Built

- **Firewall:** Protectli Vault running pfSense
- **LAN Network:** 10.10.10.0/24
- **Web App Host:** Raspberry Pi at 10.10.10.30:8081
- **New Screened Subnet:** VLAN (SCREENED_THM)
- **Client OS:** Kali Linux on the screened subnet

The screened subnet is **air-gapped by default**:

- No blanket access to LAN

- No lateral movement
 - No implicit trust
-

Key Configuration Steps (High Level)

1. Created a VLAN-based screened subnet

- VLAN interface (SCREENED_THM) with its own IP range
- Parent interface left rule-free (acts only as a trunk)

2. Applied firewall rules on the VLAN interface

- Allowed **only TCP port 8081**
- Source: SCREENED_THM subnet
- Destination: 10.10.10.30
- Everything else implicitly denied

3. Verified application binding

- Confirmed the web app listens on 0.0.0.0:8081
- Ensured it is reachable beyond localhost

4. Validated routing and filtering

- ICMP (ping) confirmed network reachability
 - HTTP access tested from a screened-subnet client
-

Final Result

✅ A user on the **screened subnet** can successfully access

http://10.10.10.30:8081

❌ That same user **cannot access**:

- Other LAN hosts
- Other services on the Raspberry Pi
- The rest of the internal network

This confirms:

- VLAN segmentation is working
 - pfSense rules are correctly applied on ingress
 - Access is limited to **one approved application**
-

Why This Matters (Security Takeaway)

This design demonstrates a **real-world defensive pattern**:

“Expose services, not networks.”

Instead of trusting an entire subnet, access is:

- **Explicit**
- **Minimal**
- **Auditable**

This mirrors how production environments protect:

- Admin dashboards
- Jump hosts
- Internal web apps
- SOC tooling